

Aula 03 — Seleção de Modelos e Validação Cruzada

Curso de Aprendizado de Máquina — UFF

Prof. Gabriel Sanfins

Leitura recomendada: AME §1.4–1.5; ISLP cap. 5

Objetivos da aula

Ao final desta aula o aluno deve ser capaz de:

- explicar por que o erro de treino é um estimador *otimista* do risco;
- descrever *data splitting* (treino/validação) e estimar o risco com a parte de validação;
- definir *validação cruzada* LOOCV e *k*-dobras e relacioná-las;
- discutir o balanço viés–variância *na própria estimativa do risco* e a escolha de *k*;
- usar a validação cruzada para **selecionar hiperparâmetros** sem vazamento;
- descrever o *bootstrap* para quantificar incerteza de uma estimativa.

Em sala

Esta é a aula *operacional* mais importante do curso: é a ferramenta que usaremos para escolher *todos* os *tuning parameters* daqui em diante (λ do Lasso, k do KNN, profundidade da árvore, C da SVM...). Pergunta de abertura: “se eu ajustar um polinômio de grau 50 a 50 pontos, qual o erro no treino? E isso quer dizer que é um bom modelo?”

1 O problema: estimar o risco

Da Aula 01, o risco de uma função de predição g é $R(g) = \mathbb{E}[L(Y, g(\mathbf{X}))]$, esperança sobre uma observação *nova*. Para *selecionar* um modelo (escolher entre candidatos $g \in \mathcal{G}$, ou escolher um hiperparâmetro) precisamos comparar riscos — mas $R(g)$ é desconhecido porque depende de P . Logo, **tudo se resume a estimar bem o risco**.

1.1 Por que não usar o erro de treino?

O candidato ingênuo é o *erro quadrático médio no treino* (risco empírico):

$$\text{EQM}(g) = \frac{1}{n} \sum_{i=1}^n (Y_i - g(\mathbf{X}_i))^2. \quad (1)$$

O problema: g foi *escolhida para ajustar exatamente esses pontos*. O $\text{EQM}(1)$ é um estimador **otimista** (enviesado para baixo) de $R(g)$, e esse viés — o *otimismo* — cresce com a complexidade do modelo.

Em sala

Cuidado com a palavra, e vale escrever as duas no quadro lado a lado: o viés desta frase é o do *EQM como estimador do risco*, $\mathbb{E}[\text{EQM}(g)] - R(g)$, que *cresce* com a complexidade. Não é o viés do *modelo* da Aula 01, $\mathbb{E}_{\mathcal{D}}[\hat{r}(\mathbf{x}_0)] - r(\mathbf{x}_0)$, que *cai* com a complexidade. Um é o erro do modelo, o outro é o erro da régua que mede o modelo — e o modelo mais

flexível é o que tem viés pequeno e régua mais mentirosa. Alguém vai perguntar; melhor antecipar.

Atenção

Levado ao extremo, minimizar o erro de treino leva ao superajuste perfeito: um modelo flexível o bastante *interpola* os dados (erro de treino zero, em aritmética exata — ver a §3 da Aula prática 03 para o que o ponto flutuante faz com isso) e generaliza pessimamente. **Nunca** selecione modelos nem reporte desempenho com o erro de treino.

Observação 1.1. Penalização (AIC, BIC) é uma alternativa: estima-se $R(g) \approx \text{EQM}(g) + P(g)$, com $P(g)$ crescente na complexidade (p.ex. $P(g) \propto p/\hat{\sigma}^2$ no AIC). Funciona, mas exige um modelo probabilístico e contagem de “parâmetros” — nem sempre disponível (KNN, florestas). A validação cruzada é *agnóstica ao modelo* e por isso a preferimos.

2 Data splitting (conjunto de teste)

A ideia mais simples: separe os dados *aleatoriamente* em treinamento (p.ex. 70%) e teste (p.ex. 30%):

$$\underbrace{(\mathbf{X}_1, Y_1), \dots, (\mathbf{X}_n, Y_n)}_{\text{treinamento}}, \underbrace{(\mathbf{X}_{n+1}, Y_{n+1}), \dots, (\mathbf{X}_{n+m}, Y_{n+m})}_{\text{teste}}.$$

Daqui para a frente n é o tamanho do *treinamento* e m o do teste. Ajuste g só no treinamento e estime o risco só no teste:

$$\hat{R}(g) = \frac{1}{m} \sum_{i=n+1}^{n+m} (Y_i - g(\mathbf{X}_i))^2. \quad (2)$$

Como o teste não foi usado para estimar g , (2) é um estimador *consistente* de $R(g)$ pela lei dos grandes números. A proporção também não é arbitrária: treinar um modelo exige mais informação do que avaliá-lo, e por isso a fatia maior fica com o treinamento.

Em sala

Enfatize o *aleatoriamente*: se o banco vier ordenado por alguma variável (p.ex. por data ou por classe), uma divisão pelos índices originais cria conjuntos não representativos. Em `scikit-learn` isso é o `train_test_split(..., shuffle=True)` (padrão).

Atenção: Limitações do data splitting

1. **Desperdício de dados:** g é treinada com apenas n das $n + m$ observações que você coletou; com poucos dados isso prejudica o ajuste.
2. **Variabilidade:** a estimativa depende da divisão sorteada — outra semente dá outro número. Pode ser instável.

A validação cruzada resolve os dois usando *todo o conjunto de treinamento* — e é importante ver o que ela *não* substitui. A divisão treino/teste continua sendo o primeiro passo, e o teste continua guardado até o fim. O que a validação cruzada dispensa é um *segundo* corte, o que separaria uma parte do treino para escolher o modelo: esse papel passa a ser feito pelas dobras, girando dentro do treino.

3 Validação cruzada

3.1 Leave-one-out (LOOCV)

Deixe *uma* observação de fora por vez. Seja g_{-i} o modelo ajustado sem a i -ésima observação. O estimador é

$$\hat{R}_{\text{LOO}}(g) = \frac{1}{n} \sum_{i=1}^n (Y_i - g_{-i}(\mathbf{X}_i))^2. \quad (3)$$

Cada Y_i é predito por um modelo que *não* o viu. LOOCV é *aproximadamente não-viesado* para o risco esperado e não tem aleatoriedade de divisão. O custo aparente é alto (n ajustes), mas:

Proposição 3.1 (Atalho do LOOCV para modelos lineares). *Para um ajuste linear (MQO, Ridge, splines) com matriz de projeção $\mathbf{H} = \mathbf{X}(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top$, vale*

$$\hat{R}_{\text{LOO}} = \frac{1}{n} \sum_{i=1}^n \left(\frac{Y_i - \hat{g}(\mathbf{X}_i)}{1 - h_{ii}} \right)^2,$$

onde h_{ii} é o i -ésimo elemento diagonal de $\mathbf{H}$ (leverage) e $\hat{g}$ é ajustado com todos os dados.

Ou seja, um *único* ajuste basta. Esta é uma das fórmulas mais elegantes da área: o LOOCV “de graça”.

3.2 Validação cruzada com k dobras

Divida o conjunto de treinamento aleatoriamente em k lotes (dobras) disjuntos de tamanho $\approx n/k$, com índices $L_1, \dots, L_k$. O conjunto de teste saiu na divisão anterior e não entra em dobra nenhuma. Para cada j , treine g_{-j} sem o lote L_j e avalie nesse lote:

$$\hat{R}_{k\text{-CV}}(g) = \frac{1}{n} \sum_{j=1}^k \sum_{i \in L_j} (Y_i - g_{-j}(\mathbf{X}_i))^2. \quad (4)$$

Cada observação é usada para validação *exatamente uma vez* e para treino $k - 1$ vezes. Quando $k = n$ recuperamos o LOOCV (3). Valores típicos: $k = 5$ ou $k = 10$.

Em sala

Alguém que tenha lido o [ISLP] vai estranhar: a fórmula (5.3) de lá é $\text{CV}_{(k)} = \frac{1}{k} \sum_j \text{EQM}_j$, a média dos EQM das dobras. É a mesma coisa — agrupando (4) por dobra sai $\sum_j (|L_j|/n) \text{EQM}_j$, a mesma média *ponderada pelo tamanho das dobras*. Coincidem exatamente quando k divide n . A nossa pesa cada observação igualmente; a do [ISLP], cada dobra — e é a do [ISLP] que o `cross_val_score` calcula. Quando k não divide n , a §6 da Aula prática 03 mede a diferença: 0,9% com $n = 50$, $k = 7$, e o grau escolhido muda em 9 de 300 amostras. Daí a regra prática: com n pequeno, escolha um k que divida n .

Ideia central

A validação cruzada estima o risco *de um procedimento* (“ajustar este tipo de modelo a dados como estes”), não de um g específico. Por isso, depois de escolher o hiperparâmetro por CV, **reajustamos o modelo final usando todos os dados**.

3.3 Viés, variância e a escolha de k

O argumento clássico para escolher k é ele mesmo um balanço viés-variância, mas da *estimativa do risco*:

- **k grande (LOOCV):** cada g_{-j} treina com $\approx n$ dados $\Rightarrow$ *pouco viés*. Mas os n modelos são quase idênticos e suas predições muito correlacionadas — e a média de quantidades correlacionadas não se beneficia da lei dos grandes números como a de quantidades independentes. O argumento conclui: *variância alta*. O custo também é alto (salvo o atalho da Prop. 3.1).
- **k pequeno (p.ex. 5):** cada g_{-j} treina com menos dados $\Rightarrow$ *mais viés*, no sentido *pessimista*: o procedimento avaliado não é bem o que você vai usar no final. Em troca, as dobras são mais independentes.

Em sala

Pergunte à turma o que acontece com o viés quando k cresce, antes de responder. O caminho é: cada dobra treina com $n - n/k$ observações, então k maior aproxima o procedimento avaliado daquele que será usado no fim — e o viés, que é pessimista, encolhe. A simulação da versão do aluno mede: 0,43 em $k = 2$ contra 0,006 em $k = 10$.

Daí a conclusão prática: de $k = 5$ em diante não há praticamente nada a ganhar, e o custo continua subindo linearmente. Por isso $k = 5$ ou $k = 10$.

Atenção: A regra de ouro contra vazamento

Toda etapa que “aprende” algo dos dados — padronização, seleção de variáveis, escolha de λ — deve ocorrer *dentro* de cada dobra, usando apenas o treino daquela dobra. Padronizar com a média/desvio de todo o conjunto antes da CV é *data leakage* e produz estimativas otimistas. (É por isso que usamos *pipelines*, Aula 06.)

3.4 Selecionando hiperparâmetros

O uso central: para cada valor de um hiperparâmetro θ (p.ex. λ do Lasso), calcule $\hat{R}_{k-CV}(\theta)$ e escolha $\hat{\theta} = \arg \min_{\theta} \hat{R}_{k-CV}(\theta)$.

Ideia central: o protocolo, em quatro passos

Junte as peças e o procedimento inteiro fica assim:

1. **separe** o conjunto de teste (p.ex. 30%) e guarde-o;
2. **escolha** um ou mais bons candidatos por validação cruzada *dentro do treino*;
3. **reajuste** o(s) candidato(s) em *todo* o conjunto de treinamento — as dobras serviram para escolher, não para entregar o modelo;
4. **meça** o desempenho no conjunto de teste.

O passo 3 é o que quase todo mundo esquece, e é ele que faz a CV valer a pena: o modelo entregue usa *todas* as observações de treino, não $k - 1$ delas.

Atenção: Regra “1-EP”

Como $\hat{R}_{k-CV}$ tem erro padrão, muitas vezes prefere-se a *regra de um erro-padrão*: escolher o modelo *mais simples* cujo risco estimado esteja a no máximo um erro-padrão do mínimo. Favorece parcimônia (é o `lambda.1se` do `glmnet`).

Atenção: Validação *aninhada*

Se você usa CV para *escolher* θ e para *reportar* o desempenho, precisa de **CV aninhada**: um laço externo estima o desempenho, um laço interno escolhe θ . Usar a mesma CV para as duas coisas subestima o risco.

4 O bootstrap

A validação cruzada estima o *risco*; o **bootstrap** estima a *incerteza* (variância, viés, intervalos) de praticamente qualquer estatística $\hat{\theta}$ calculada a partir dos dados.

Ideia central

Não conhecemos P , mas temos a amostra. O bootstrap usa a *distribuição empírica* (reamostrar dos dados *com reposição*) como substituta de P , imitando o ato de coletar novas amostras.

Algoritmo. Para $b = 1, \dots, B$:

1. sorteie n observações *com reposição* da amostra original (amostra bootstrap $\mathcal{D}^{*b}$);
2. calcule a estatística $\hat{\theta}^{*b}$ em $\mathcal{D}^{*b}$.

A dispersão de $\{\hat{\theta}^{*1}, \dots, \hat{\theta}^{*B}\}$ estima a distribuição amostral de $\hat{\theta}$; por exemplo, $\widehat{EP}(\hat{\theta})$ é o desvio-padrão dessas réplicas, e um intervalo de confiança de 95% pode ser dado pelos percentis 2,5% e 97,5%.

Observação 4.1. Cada amostra bootstrap deixa de fora, em média, $\approx 1/e \approx 37\%$ das observações originais (as *out-of-bag*). Essa ideia reaparece no *bagging* e nas florestas aleatórias (Aula 05), onde as observações OOB dão uma estimativa de risco “de graça”.

5 Prática em Python

```

1 from sklearn.model_selection import (train_test_split, cross_val_score, KFold,
2                                     GridSearchCV)
3 from sklearn.pipeline import make_pipeline
4 from sklearn.preprocessing import StandardScaler
5 from sklearn.linear_model import Ridge
6 from sklearn.metrics import mean_squared_error
7
8 # 0. O PRIMEIRO passo é separar o teste. Dobra nenhuma vai toca-lo.
9 X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=0)
10
11 modelo = make_pipeline(StandardScaler(), Ridge(alpha=1.0))
12 dobras = KFold(5, shuffle=True, random_state=0)
13
14 # 1. CV DENTRO DO TREINO, para estimar o risco de UM procedimento
15 # (MSE -> negativo no sklearn)
16 scores = cross_val_score(modelo, X_tr, y_tr, cv=dobras,
17                           scoring="neg_mean_squared_error")
18 print("risco estimado:", -scores.mean(), "+/-", scores.std())
19
20 # 2. Selecionar o hiperparametro lambda (=alpha) por CV, SEM vazamento
21 # (o StandardScaler é reajustado dentro de cada dobra)
22 grade = {"ridge_alpha": [0.01, 0.1, 1, 10, 100]}
23 busca = GridSearchCV(modelo, grade, cv=dobras, scoring="neg_mean_squared_error")
24 busca.fit(X_tr, y_tr) # o teste continua guardado

```

```
25 print("melhor alpha:", busca.best_params_)
26
27 # 3. O vencedor ja saiu reajustado em TODO o treino (refit=True, o padrao).
28 # So agora o teste e tocado, uma unica vez.
29 print("EQM no teste:", mean_squared_error(y_te, busca.predict(X_te)))
```

O notebook Aula prática 03 implementa as k dobras à mão, confere o atalho do LOOCV pela alavancagem, mede o efeito de rodar KFold sem `shuffle`, roda o protocolo de quatro passos contra a resposta conhecida e fecha escolhendo o λ de uma Ridge no `superconductivity.csv`.

Resumo

- O erro de treino é otimista; estimar o risco exige dados *não* usados no ajuste.
- *Data splitting*: simples, mas desperdiça dados e é instável.
- *LOOCV* (3): quase não-viesado; atalho fechado para modelos lineares (Prop. 3.1).
- *k-dobras* (4): usa toda a amostra; $k = 5$ ou 10 equilibram viés/variância/custo.
- Use CV para escolher *tuning parameters*; cuidado com *vazamento* (faça tudo dentro da dobra) e com CV aninhada para reportar desempenho.
- *Bootstrap*: reamostragem com reposição para medir incerteza de estimativas; conecta com OOB e *bagging*.

Leitura recomendada. [AME] §1.4 (função de risco e Teorema 1), §1.5.1 (*data splitting* e validação cruzada); [ISLP] Capítulo 5 (§5.1 validação cruzada, §5.2 *bootstrap*).